Testabschlussbericht

Geschenke-Manager

Projekt: Software Engineering (ISEF01)

Meilenstein 4, Liefergegenstand LG-09

Datum: 6. November 2026

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Zweck 1

1.2 Testgegenstand 1

1.3 Aufbau 1

2 Testumfang und Testumgebung **1**

2.1 Teststufen 1

2.2 Testumgebungen 2

3 Ergebnisse im Überblick **2**

4 Ergebnisse je Qualitätsziel **3**

4.1 QZ-01 Funktionale Eignung 3

4.2 QZ-02 Leistungseffizienz 3

4.3 QZ-03 Kompatibilität 4

4.4 QZ-04 Benutzbarkeit und Barrierefreiheit 4

4.5 QZ-05 Zuverlässigkeit 5

4.6 QZ-06 Sicherheit 5

4.7 QZ-07 Wartbarkeit 6

4.8 QZ-08 Übertragbarkeit 6

4.9 QZ-09 Zeitkorrektheit 7

5 Rückverfolgbarkeit **7**

6 Testprotokolle **8**

7 Restrisiken **9**

7.1 Beobachtungen aus den Systemtests 9

7.2 Grenzen der Tests 9

8 Fazit und Empfehlung **9**

Tabellenverzeichnis

1 Teststufen und Werkzeuge 1

2 Ergebnis je Qualitätsziel 2

3 Antwortzeiten der Listen (QZ-02) 3

4 Testfälle je Anforderung im Lauf auf dem Release-Stand 7

5 Testläufe und Protokolle 8

1 Einleitung

1.1 Zweck

Dieser Bericht schließt die Tests des Geschenke-Managers für Meilenstein 4 ab. Er hält fest, welcher Softwarestand geprüft wurde, mit welchen Mitteln und mit welchem Ergebnis. Grundlage für die Bewertung sind die Qualitätsziele QZ-01 bis QZ-09 und die Teststrategie aus Meilenstein 3. Für jedes Ziel nennt der Bericht den Nachweis, den Messwert und die Bewertung. Am Ende stehen die offenen Fehler, die bekannten Grenzen der Tests und eine Empfehlung zur Abnahme.

1.2 Testgegenstand

Geprüft wurde der Release-Stand für Meilenstein 4. Das Backend ist gegenüber `57b9a7a` unverändert, das Frontend gegenüber `12046b5`. Auf diesen Ständen liefen die letzten CI-Läufe für Backend und Frontend, ihre Ergebnisse sind deshalb auf den Release-Stand übertragbar.

1.3 Aufbau

Kapitel 2 beschreibt Teststufen und Testumgebungen. Kapitel 3 fasst das Ergebnis für alle Qualitätsziele zusammen, Kapitel 4 begründet es Ziel für Ziel. Kapitel 5 zeigt die Rückverfolgbarkeit von den Anforderungen zu den Testfällen, Kapitel 6 listet die Testprotokolle. Kapitel 7 nennt offene Fehler und Restrisiken, Kapitel 8 schließt mit der Empfehlung.

2 Testumfang und Testumgebung

2.1 Teststufen

Tabelle 1: Teststufen und Werkzeuge

Stufe	Inhalt	Werkzeug
Statische Prüfung	Linter, Formatierung und Typprüfung für Backend und Frontend, Vollständigkeit der Datenbankmigrationen, Aktualität der generierten API-Typen	Ruff, Pyright, Alembic, ESLint, svelte-check
Backend, Unit und Integration	730 Testfälle gegen eine echte PostgreSQL-Datenbank in einem Testcontainer. Jeder Testfall trägt die geprüften Anforderungs-IDs.	pytest, pytest-cov, Testcontainers
Frontend, Unit und Komponenten	45 Testdateien für Serverlogik, Formularaktionen und Svelte-Komponenten; Komponententests laufen in Chromium. Der HTML-Export wird mit dem Nu Html Checker geprüft.	Vitest, Playwright, vnu
Systemtest	Manuelle Durchläufe der Kernabläufe und der Ideenliste im Browser, jeder Schritt mit Screenshot belegt	Google Chrome
Lasttest	Antwortzeiten aller Listen mit synthetischem Testbestand	`qz02_measure.py`
Betriebstest	Absturz des Schedulers und Neustart von Docker mit laufendem Stack	Docker Compose
Sicherheitsprüfung	Abhängigkeiten, Container-Images und Repository auf Schwachstellen und Secrets	pip-audit, pnpm audit, Trivy, Gitleaks

2.2 Testumgebungen

CI. Alle automatisierten Prüfungen laufen bei jedem Push und jedem Pull Request in GitHub Actions auf `ubuntu-latest`. Die Werkzeugversionen sind über mise festgelegt, die Abhängigkeiten über Lockfiles.

Lokaler Testrechner. Lenovo 83D3 mit AMD Ryzen 7 8845HS (8 Kerne, 16 Threads) und 32 GB RAM, Windows 11 (10.0.26200), Docker Desktop mit Engine 29.5.3. Auf diesem Rechner liefen der Lasttest und die Wiederholung der Backend-Tests auf dem Release-Stand.

Systemtests und Betriebstest. macOS mit Google Chrome und Docker Desktop (Engine 29.9.2).

Testdaten. Alle Tests arbeiten mit synthetischen Daten. E-Mail-Adressen liegen unter der reservierten Domain `example.org`. Der Testbestand für den Lasttest entsteht mit `mise -C backend run seed` und umfasst je Konto 100 Personen, 1.000 Geschenkideen und 1.000 Beschenkungen.

3 Ergebnisse im Überblick

Tabelle 2: Ergebnis je Qualitätsziel

Ziel	Merkmal	Kernaussage	Bewertung
QZ-01	Funktionale Eignung	Alle MUSS-Anforderungen F-01 bis F-17 haben Testfälle, alle 730 Backend-Testfälle sind bestanden	erfüllt
QZ-02	Leistungseffizienz	p95 aller Listen zwischen 17 und 246 ms bei einem Grenzwert von 2 s, gemessen an der API	auf API-Ebene erfüllt
QZ-03	Kompatibilität	Generierte API-Typen aktuell, svelte-check ohne Fehler, HTML-Export ohne Befund im Nu Html Checker	erfüllt
QZ-04	Benutzbarkeit und Barrierefreiheit	Löschbestätigungen und verständliche Fehlermeldungen belegt	teilweise nachgewiesen
QZ-05	Zuverlässigkeit	Scheduler-, Ausfall- und Neustarttests bestanden	erfüllt
QZ-06	Sicherheit	Autorisierungstests bestanden, Image-Scan und Secret-Scan ohne Befund, keine blockierenden Audit-Befunde	erfüllt
QZ-07	Wartbarkeit	Linter und Typprüfung fehlerfrei, Abdeckung 96,5 %, 72 von 79 Merges mit Freigabe	teilweise erfüllt
QZ-08	Übertragbarkeit	Start mit `docker compose up` mehrfach belegt, Kernabläufe in Chrome bestanden	teilweise nachgewiesen
QZ-09	Zeitkorrektheit	72 Testfälle mit fester Test-Uhr bestanden	erfüllt

Die Bewertung „teilweise“ bedeutet, dass die geprüften Teilkriterien erfüllt sind, aber nicht alle Teilkriterien des Ziels in diesem Testzyklus geprüft wurden. Kein geprüftes Kriterium ist durchgefallen.

4 Ergebnisse je Qualitätsziel

4.1 QZ-01 Funktionale Eignung

Kriterium	Alle MUSS-Anforderungen sind umgesetzt und durch mindestens einen Testfall abgedeckt, 100 % der zugehörigen Testfälle sind bestanden. Berechnungen sind korrekt. Eine Geschenkidee wird in einem Schritt zur Beschenkung.
Nachweis	Rückverfolgbarkeitsmatrix aus dem Testlauf, Backend-Tests, Frontend-Tests
Ergebnis	erfüllt

728 der 730 Backend-Testfälle nennen über die Markierung `@pytest.mark.requirement` die Anforderungen, die sie prüfen. Aus jedem vollständigen Lauf erzeugt `tests/conftest.py` die Matrix `docs/generated/traceability.md`. Im Lauf auf dem Release-Stand waren alle 730 Testfälle bestanden. Jede der 17 MUSS-Anforderungen hat mindestens 5 Testfälle, die Verteilung zeigt Tabelle 4 in Kapitel 5.

Die Beispiele aus dem Kriterium sind eigens geprüft. `test_lists_exactly_next_month` und `test_december_lists_january` prüfen, dass die Geburtstagsübersicht genau die Personen des Folgemonats enthält, auch über den Jahreswechsel. `test_feb_29_is_in_the_february_mail` prüft den 29. Februar. Dass eine Idee mit einer einzigen Anfrage zur Beschenkung wird und dabei nicht kopiert, sondern referenziert wird, prüft `tests/test_giftings.py`.

4.2 QZ-02 Leistungseffizienz

Kriterium	Listen mit 100 Personen, 1.000 Geschenkideen und 1.000 Beschenkungen werden innerhalb von 2 s angezeigt. Nach einem Aufwärmlauf 30 Messungen, der 95. Perzentilwert muss höchstens 2 s betragen.
Nachweis	Lasttest mit `backend/scripts/qz02_measure.py` am 03.11.2026 auf dem lokalen Testrechner, Stand `d91d367`
Ergebnis	auf API-Ebene erfüllt

Gemessen wurde mit dem kompletten Stack in Docker Compose und dem Seed-Testbestand, direkt gegen die API unter `localhost:8000`, ohne andere Last auf dem Rechner. Das Konto `test1@example.org` hatte 100 Personen, 23 Anlässe, 1.000 Geschenkideen, 1.000 Beschenkungen und rund 1.500 Aufgaben. Der p95 ist nach Nearest Rank bestimmt, also der 29. von 30 sortierten Werten.

Tabelle 3: Antwortzeiten der Listen (QZ-02)

Endpunkt	Median	p95	Max	p95 $\leq$ 2 s
`/full-list`	134 ms	246 ms	275 ms	ja
`/people?limit=200`	13 ms	17 ms	22 ms	ja
`/occasions?limit=200`	15 ms	17 ms	21 ms	ja
`/gifts?limit=200`	34 ms	40 ms	123 ms	ja
`/giftings?limit=200`	51 ms	126 ms	139 ms	ja
`/tasks?limit=200`	14 ms	19 ms	22 ms	ja
`/people/{id}/overview?limit=200`	48 ms	64 ms	66 ms	ja

Die Personenansicht wurde für die Person mit den meisten Einträgen gemessen, also für den ungüns-

tigsten Fall. Der langsamste Endpunkt ist die vollständige Liste (F-15) mit einem p95 von 246 ms, das ist rund ein Achtel des Grenzwerts. Die Messung erfasst die Antwortzeit der API. Die Zeit für Übertragung und Darstellung im Browser ist darin nicht enthalten, deshalb lautet die Bewertung „auf API-Ebene erfüllt“. Seit dem gemessenen Stand haben sich im Backend nur die Ideenliste, die Bildverarbeitung und das Basis-Image (Alpine, #170) geändert. Der Code der gemessenen Listen ist unverändert.

4.3 QZ-03 Kompatibilität

Kriterium	Der Frontend-Client entspricht der OpenAPI-Spezifikation des Backends, ohne Typfehler. Der HTML-Export ist valides HTML.
Nachweis	CI-Schritte „OpenAPI schema in frontend is up to date“ und „Generated API types are up to date“, svelte-check, Test `full-list/server.test.ts`
Ergebnis	erfüllt

Das Backend exportiert seine OpenAPI-Spezifikation, das Frontend erzeugt daraus seine API-Typen. Beide CI-Workflows brechen ab, sobald Spezifikation oder Typen nicht zum Code passen. Auf dem Release-Stand waren beide Schritte grün, svelte-check lief ohne Fehler. Die vollständige Liste als HTML-Dokument (F-15) prüft ein Frontend-Test mit dem Nu Html Checker, der Engine hinter validator.w3.org. Der Test verlangt eine Prüfung ohne Meldungen und war bestanden.

4.4 QZ-04 Benutzbarkeit und Barrierefreiheit

Kriterium	Erfassung einer Geschenkidee in höchstens 3 Interaktionen und unter 30 s. Löschvorgänge erfordern eine Bestätigung, ungültige Eingaben erzeugen verständliche Fehlermeldungen, wesentliche Abläufe funktionieren per Tastatur, Zustände werden nicht nur durch Farbe vermittelt. Lighthouse-Accessibility-Wert $\geq$ 90.
Nachweis	Systemtest der Kernabläufe, Frontend-Tests zu Fehlermeldungen und Dialogen, Backend-Validierungstests
Ergebnis	teilweise nachgewiesen

Löschbestätigung. Personen, Geschenkideen, Aufgaben, Notizen, Links und Bilder werden erst nach einem Bestätigungsdialog gelöscht, der den betroffenen Eintrag nennt, etwa „Kochkurs wird mit allen zugehörigen Daten endgültig gelöscht.“ Die Kontolöschung verlangt zusätzlich ein angehaktes Kontrollkästchen. Der Systemtest belegt das in Schritt 16 (Screenshot `16-konto-loeschen-bestaetigung.png`).

Fehlermeldungen. Das Backend lehnt ungültige Eingaben mit einer Fehlerbeschreibung je Feld ab, geprüft unter anderem durch `test_invalid_person_is_rejected` und `test_register_validates_password_length`. Die Frontend-Tests prüfen, dass Dialoge bei Fehlern offen bleiben, die Eingaben behalten und die Meldung beim Feld anzeigen, etwa „keeps the dialog open with field errors and does not call the backend“. Im Systemtest erscheint bei einer Anmeldung mit dem gelöschten Konto die Meldung „E-Mail-Adresse oder Passwort ist falsch.“ (Schritt 18).

Bewertung. Die beiden Teilkriterien Löschbestätigung und Fehlermeldungen sind nachgewiesen. Die Erfassungszeit mit Testpersonen, die Tastaturbedienung, die Farbunabhängigkeit und der Lighthouse-Wert wurden in diesem Testzyklus nicht gemessen. QZ-04 ist deshalb nur teilweise nachgewiesen.

4.5 QZ-05 Zuverlässigkeit

Kriterium	Benachrichtigungen entstehen zum vorgesehenen Termin. Wiederholte oder parallele Scheduler-Läufe erzeugen keine zusätzlichen Versandaufträge. Unklare Versandergebnisse werden protokolliert, nicht erneut versendet. Ein Ausfall des Mailversands beeinträchtigt die übrigen Funktionen nicht. Container starten nach einem Neustart automatisch.
Nachweis	40 Testfälle mit der Kennung QZ-05, Neustart- und Absturztest (`docs/ms4/neustart-test/`)
Ergebnis	erfüllt

Die Planung der Benachrichtigungen wird mit einer festen Test-Uhr geprüft, sodass jeder Termin gezielt angesteuert werden kann. Eigene Testfälle starten zwei Scheduler-Läufe parallel und prüfen, dass jede Mail nur einmal geplant und versendet wird.
`test_unclear_result_is_logged_and_not_resent` prüft, dass ein unklares Versandergebnis protokolliert und nicht erneut versendet wird. `test_scheduler_rides_out_smtp_outage_and_sends_once_back` simuliert einen Ausfall des SMTP-Servers: Die Mail bleibt mit Fehlervermerk geplant und wird nach dem Ende des Ausfalls genau einmal zugestellt. Dass der API-Prozess weder Scheduler noch Mailversand lädt und deshalb von einem Ausfall unberührt bleibt, prüft `test_api_process_loads_no_scheduler_or_mail`. Alle diese Testfälle waren bestanden.

Den automatischen Neustart belegt ein skriptgesteuerter Lauf mit dem kompletten Compose-Stack. Zuerst wurde der Scheduler-Prozess mit `SIGKILL` beendet. Docker meldete `die exitCode=137` und startete den Container in derselben Sekunde neu, der nächste Scheduler-Lauf war eine Sekunde später abgeschlossen. Danach wurde Docker Desktop neu gestartet. Alle fünf Dienste liefen nach acht Sekunden wieder, ohne `docker compose up`. Health-Endpunkte und Anmeldeseite antworteten mit Status 200, der erste Scheduler-Lauf war erfolgreich.

4.6 QZ-06 Sicherheit

Kriterium	Nutzende greifen nur auf eigene Daten zu, einzige Ausnahme sind Inhalte über einen gültigen Teilen-Link. Passwörter und Tokens werden nur gehasht gespeichert. Teilen-Links sind nicht erratbar, optional befristet und deaktivierbar. Keine Secrets im Repository, nur synthetische Daten.
Nachweis	Autorisierungstests, pip-audit, pnpm audit, Trivy, Gitleaks, Systemtest der Teilen-Links
Ergebnis	erfüllt

Mandantentrennung. Zu jeder Ressource gibt es Testfälle, in denen ein zweites Konto auf die Daten des ersten zugreift. Die Hilfsfunktion `assert_invisible_to_other_account` prüft dabei Lesen, Ändern und Löschen und erwartet jeweils Status 404, damit fremde Daten nicht einmal ihre Existenz verraten. Sie wird in 12 Testdateien verwendet.

Passwörter, Tokens und Teilen-Links. Passwörter werden mit Argon2id gehasht, Sitzungs- und Link-Tokens nur als Hash gespeichert. Im Systemtest öffnete ein Browser-Kontext ohne Anmeldung einen Teilen-Link und sah nur die freigegebenen Ideen, ohne Formulare und mit den Kopfzeilen `Referrer-Policy: no-referrer`, `X-Robots-Tag: noindex` und `Cache-Control: no-store`. Schreibversuche über den Link mit PUT, POST, PATCH und DELETE wurden alle mit Status 405 abgewiesen, die Daten blieben unverändert. Nach der Kontolöschung lieferte derselbe Link Status 404.

Scans. Auf dem Release-Stand lief pip-audit (`--strict`) ohne Befund. `pnpm audit --audit-level=high` meldete keine hohen oder kritischen Befunde, aber einen niedrigen und einen mittleren. Beide liegen in Werkzeugen für Build und Dokumentation (`deepmerge` über `vite-plugin-svelte`, `minimatch` über `typedoc`) und blockieren den Lauf nicht. Trivy prüfte beide Container-Images auf Schwachstellen der Stufen hoch und kritisch, ebenfalls ohne Befund; die Ausnahmeliste `.trivyignore` ist leer. Gitleaks fand im Repository keine Secrets.

4.7 QZ-07 Wartbarkeit

Kriterium	Linter und Typprüfung fehlerfrei. Jeder Merge auf `main` nach Code-Review. Zeilenabdeckung des Backends $\geq$ 70 %, jeder API-Endpunkt mit mindestens einem automatisierten Test. Frontend und Backend getrennt bau- und testbar.
Nachweis	CI-Läufe, lokaler Testlauf mit pytest-cov, Pull-Request-Historie auf GitHub
Ergebnis	teilweise erfüllt

Statische Prüfung und Abdeckung. Ruff, Pyright, ESLint und svelte-check liefen auf dem Release-Stand ohne Fehler. Die Zeilenabdeckung des Backends lag bei 96,5 %. Die CI bricht unter 70 % ab (`--cov-fail-under=70`). Die Abdeckung der Endpunkte wurde über die Abdeckungsdaten geprüft: Alle 66 Endpunkte der API wurden im Testlauf ausgeführt, für jeden gibt es also mindestens einen automatisierten Test.

Getrennte Bau- und Testbarkeit. Backend und Frontend haben eigene Workflows, eigene Lockfiles und eigene Docker-Images, die getrennt gebaut und gescannt werden.

4.8 QZ-08 Übertragbarkeit

Kriterium	Das Gesamtsystem startet aus leerem Zustand mit `docker compose up` anhand der Betriebsdokumentation. Kernfunktionen sind in Chrome, Firefox und Safari sowie auf Smartphone-Breite nutzbar.
Nachweis	Lasttest und Betriebstest mit Compose-Stack, Systemtest der Kernabläufe, Komponententests in Chromium
Ergebnis	teilweise nachgewiesen

Start mit Docker Compose. Der komplette Stack aus Datenbank, Migration, API, Scheduler, Frontend und Mailpit wurde unabhängig voneinander auf zwei Rechnern mit `docker compose up` gestartet: unter Windows für den Lasttest und unter macOS für den Betriebstest. Die Betriebsdokumentation im `README.md` beschreibt diesen Weg Schritt für Schritt.

Kernabläufe im Browser. Der Systemtest vom 03.11.2026 lief in Google Chrome durch alle Kernabläufe aus der Teststrategie: registrieren, Person anlegen, Idee anlegen, Beschenkung planen und als verschenkt markieren, Teilen-Link ohne Anmeldung öffnen, Schreibversuche über den Link abweisen und Konto löschen. Alle 19 Schritte hatten das erwartete Ergebnis. Ein zweiter Systemtest vom 04.11.2026 prüfte die Ideenliste (F-16) mit drei Konten, ebenfalls ohne Abweichung. Die Komponententests des Frontends laufen in jeder CI in Chromium.

Bewertung. Belegt sind der Start mit Docker Compose und die Kernabläufe in Chrome beziehungsweise Chromium. Firefox, Safari und die Smartphone-Breite wurden in diesem Testzyklus nicht geprüft, der Start erfolgte durch Teammitglieder. QZ-08 ist deshalb nur teilweise nachgewiesen.

4.9 QZ-09 Zeitkorrektheit

Kriterium	Technische Zeitpunkte in UTC, Geburtstage, Anlasstermine und Verschenkdaten als reine Kalenderdaten. Benachrichtigungen berücksichtigen `Europe/Berlin`, Zeitumstellungen verschieben keinen Anlass.
Nachweis	72 Testfälle mit der Kennung QZ-09, alle mit fester Test-Uhr
Ergebnis	erfüllt

Die Anwendung bezieht die aktuelle Zeit über eine austauschbare Zeitquelle (`core/clock.py`), die Tests ersetzen sie durch eine feste Uhr. Damit prüfen sie gezielt den Jahreswechsel (`test_december_lists_january`), den 29. Februar (`test_feb_29_is_in_the_february_mail`) und die Zeitumstellung. `test_summer_time_send_hour` prüft zum Beispiel, dass die Geburtstagsübersicht im Sommer um 8 Uhr Berliner Zeit geplant wird, also um 6 Uhr UTC. `test_window_on_dst_change_day` prüft das Erinnerungsfenster am Tag der Zeitumstellung. Alle Testfälle waren bestanden.

5 Rückverfolgbarkeit

Tabelle 4 verdichtet die Matrix `docs/generated/traceability.md` aus dem Lauf auf dem Release-Stand. Ein Testfall kann mehrere Anforderungen prüfen und zählt dann bei jeder mit. Die vollständige Matrix mit jedem einzelnen Testfall liegt im Repository.

Tabelle 4: Testfälle je Anforderung im Lauf auf dem Release-Stand

ID	Anforderung	Testfälle	bestanden
F-01	Registrieren, anmelden, abmelden	16	16
F-02	Personen verwalten	21	21
F-03	Feste und eigene Anlässe	96	96
F-04	Unabhängige Objekte, n:m-Verknüpfungen	39	39
F-05	Ideen schnell erfassen	41	41
F-06	Beschenkung mit Status, Verschenken, Nachtragen	57	57
F-07	Notizen, Links und Bilder	149	149
F-08	Aufgaben je Beschenkung	35	35
F-09	Personenübersicht	9	9
F-10	Geburtstage im Folgemonat	21	21
F-11	Weihnachtsübersicht	28	28
F-12	Erinnerung zwei Wochen vorher	23	23
F-13	Versand protokollieren, keine Doppelung	76	76
F-14	Teilen-Link nur zum Lesen	42	42
F-15	Gesamtliste als HTML	5	5
F-16	Weitere Ideen über die Ideenliste	52	52
F-17	Konto vollständig löschen	16	16
Q-01	Mandantentrennung	111	111
Q-02	Benachrichtigungszuverlässigkeit	52	52
Q-03	Sicherheit	68	68
Q-04	Datenschutz	10	10

ID	Anforderung	Testfälle	bestanden
Q-05	Leistung	22	22
Q-06	Wartbarkeit	3	3
Q-07	Barrierefreiheit	0	0
Q-08	Zeitkorrektheit	45	45
Q-09	Nachvollziehbarkeit	1	1

Q-07 Barrierefreiheit betrifft die Oberfläche und hat deshalb keine Backend-Testfälle; ihr Stand steht bei QZ-04. Die SOLL-Anforderungen F-18 und F-19 wurden nicht umgesetzt und haben deshalb keine Testfälle. Die Vorschlagslogik nach Kategorie und Preisklasse ist im Backend vorhanden und getestet, im Frontend aber nicht eingebunden; F-16 wird über die Ideenliste erfüllt.

6 Testprotokolle

Tabelle 5: Testläufe und Protokolle

Lauf	Datum, Stand	Inhalt und Fundstelle	Ergebnis
CI backend, Lauf 163	05.11.2026, `57b9a7a`	Ruff, Pyright, Migrationen, Tests mit Abdeckung, OpenAPI-Abgleich, pip-audit. GitHub Actions, Run 42045127208	bestanden
CI frontend, Lauf 139	05.11.2026, `12046b5`	Typen-Abgleich, ESLint, svelte-check, Tests mit Chromium, Build, pnpm audit. Run 42046270432	bestanden
CI docker, Lauf 206	05.11.2026, `12046b5`	Bau beider Images, Trivy-Scan. Run 42046270437	bestanden
CI gitleaks, Lauf 226	06.11.2026, `fc28c35`	Secret-Scan des Release-Stands. Run 42203717511	bestanden
Backend-Tests lokal	06.11.2026, `fc28c35`	Wiederholung von Ruff, Pyright und pytest mit Abdeckung auf dem Release-Stand, lokaler Testrechner. 730 Testfälle, Laufzeit 1:47 min	bestanden
Lasttest	03.11.2026, `d91d367`	QZ-02, Tabelle 3. Protokoll in Issue #55	bestanden
Systemtest Kernabläufe	03.11.2026, `91da04b`	19 Schritte mit Screenshots, `docs/ms4/e2e-test-screenshots/`	bestanden
Systemtest Ideenliste	04.11.2026, `cf6c9d2`	5 Schritte mit Screenshots, `docs/ms4/e2e-ideenliste-screenshots/`	bestanden
Neustart- und Absturztest	04.11.2026, `d85841c`	Skriptprotokoll, `docs/ms4/neustart-test/`	bestanden

7 Restrisiken

7.1 Beobachtungen aus den Systemtests

Teilen-Link zeigt verschenkte Ideen. Die geteilte Ansicht zeigt alle Ideen, die mit der Person verknüpft sind, auch eine bereits verschenkte. Die Personenseite zählt dieselbe Idee nicht mehr als offene Idee. Für den Zweck des Links, anderen Ideen für eine Person zu zeigen, ist das irreführend, aber kein Verstoß gegen eine Anforderung.

7.2 Grenzen der Tests

- Der Neustarttest lief nur lokal mit Docker Desktop, nicht mit einem echten Neustart des Rechners. Den Fall, dass die Datenbank beim ersten Scheduler-Lauf noch nicht bereit ist, deckt er nicht ab, weil Docker alle Dienste in derselben Sekunde gestartet hat.
- Die Systemtests sind manuelle Durchläufe in einem Browser. Sie laufen nicht automatisch bei jeder Änderung.
- Der Namensfilter der Ideenliste erkennt nur Namen angelegter Personen als ganze Wörter. Gebeugte Formen wie „Lenas“ oder Namen ohne angelegte Person bleiben sichtbar. Die Titel sind Freitext.

8 Fazit und Empfehlung

Der Release-Stand `fc28c35` besteht alle automatisierten Prüfungen. Alle 730 Backend-Testfälle und alle Frontend-Tests sind bestanden, jede MUSS-Anforderung ist durch Testfälle abgedeckt, und die Sicherheits-Scans zeigen keine blockierenden Befunde; der Frontend-Audit meldet einen niedrigen und einen mittleren Befund. Die Systemtests der Kernabläufe und der Ideenliste liefen ohne Abweichung, der Betriebstest belegt den automatischen Neustart. Kein Test ist durchgefallen, und es gibt keinen offenen Fehler, der eine Anforderung verletzt.

Das Testteam empfiehlt deshalb die Abnahme des Release-Stands `fc28c35`. Einschränkungen bestehen bei den Nachweisen: QZ-02 ist nur auf API-Ebene gemessen, QZ-04 und QZ-08 sind nur teilweise nachgewiesen, und bei QZ-07 wurden nicht alle Merges vor dem Zusammenführen freigegeben.